

Uke 42:

Sortering:

- For å sortere radene i resultatet fra en SELECT-spørring, kan vi bare legge ORDER BY <kolonner> på slutten av spørringen
- hvor <kolloner> er en liste med kolonner
- For eksempel, for å sortere alle produkter etter pris:
SELECT product_name, unit_price
FROM products
ORDER BY unit_price;
- Eksempelet sorterer altså fra lavest til høyest unit_price for navnet i produktet.
- Sorteringen er gjort i henhold til typens naturlige ordning
- Sorteringen er gjort i henhold til typens naturlige ordning:
 - Tall: verdi
 - Tekst: alfabetisk
 - Tidspunkter: kronologisk
 - Osv.
- ORDER BY-klausulen kommer alltid etter WHERE-klausulen (trenger ikke ha en WHERE-klausul)

Sortere på flere kolonner og reversering:

- Standard-ordningen er fra minst til størst
- For å reversere ordningen trenger man bare legge til DESC (kort for «descending») etter kolonnenavnet.
- Med flere kolonner i ORDER BY vil radene ordnes først i henhold til den første kolonnen, så i henhold til den andre kolonnen for de med like verdier i den første, osv.
- For eksempel, for å sortere drikkevarer først på pris, og så på antall på lager, begge i nedadgående rekkefølge:
SELECT product_name, unit_price, units_in_stock
FROM products
ORDER BY unit_price DESC, units_in_stock DESC;

Begrense antall rader i resultatet:

- Når vi gjør spørringer mot store tabeller får vi ofte mange svar
- Av og til er vi ikke interessert i alle svarene
- For å begrense antall rader kan vi bruke LIMIT
- For eksempel, for å velge ut de 5 dyreste produktene:
SELECT product_name, unit_price
FROM products

ORDER BY unit_price DESC

LIMIT 5;

- LIMIT-klausulen kommer alltid til sist

Eksempel: Finn navn og pris på produktet med lavest pris

Ved `min`-aggregering og tabell-delspørring

```
SELECT p.product_name, p.unit_price
FROM products AS p INNER JOIN
    (SELECT min(unit_price) AS minprice FROM products) AS h
ON p.unit_price = h.minprice;
```

Ved `min`-aggregering og verdi-delspørring

```
SELECT product_name, unit_price
FROM products
WHERE unit_price = (SELECT min(unit_price) FROM products);
```

Ved `ORDER BY` og `LIMIT` 1

```
SELECT product_name, unit_price
FROM products
ORDER BY unit_price
LIMIT 1;
```

Hoppe over rader:

- Av og til ønsker man å hoppe over rader
- Dette er nyttig dersom man ønsker å presentere resultater i grupper
- F.eks. slik som søkeresultater fra en søkemotor, man får presentert én og én side som resultater
- Dette gjøres med `OFFSET`-klausulen
- Dersom vi ønsker å vise 10 og 10 produkter av gangen, sortert etter pris, kan man kjøre:

```
SELECT product_name, unit_price
FROM products
ORDER BY unit_price DESC
LIMIT 10
```

`OFFSET <sidetall*10>;` — Først 0, så 10, så 20 osv.

Aggregere i grupper:

- Vi har sett hvordan vi kan aggregere over hele kolonner
- Det finnes derimot en egen klausul for å gruppere radene før man aggregerer
- Nemlig GROUP BY <kolonner>
- GROUP BY tar en liste med kolonner, og grupperer dem i henhold til likhet på verdiene i disse kolonnene
- Vi kan så bruke aggregeringsfunksjoner på hver gruppe i SELECT-klausulen
- Vi kan da også ha de grupperende kolonnene sammen med aggregatet i SELECT-klausulen
- Kun de grupperte kolonnene gir mening å ha utenfor et aggregat i SELECT

Aggregere i grupper: Eksempel

Finn gjennomsnittsprisen for hver kategori

```
SELECT Category, avg(Price) AS Averageprice
FROM Products
GROUP BY Category
```

Resultat

Products				
ProductID (int)	Name (text)	Brand (text)	Price (float)	Category (text)
0	TV 50 inch	Sony	8999	Televisions
1	Laptop 2.5GHz	Lenovo	7499	Computers
2	Laptop 8GB RAM	HP	6999	Computers
3	Speaker 500	Bose	4999	Speakers
4	TV 48 inch	Panasonic	11999	Televisions
5	Laptop 1.5GHz	iPhone	5195	Computers

Ved å kjøre den kommandoen får man:

Aggregere i grupper: Eksempel

Finn gjennomsnittsprisen for hver kategori

```
SELECT Category, avg(Price) AS Averageprice
FROM Products
GROUP BY Category
```

Resultat: Regn ut aggregatet for hver gruppe og ferdigstill

Category	Averageprice
Televisions	10499
Computers	6731
Speakers	4999

Aggregering i grupper: Eksempel 1

Finn antall produkter per bestilling

```
SELECT order_id, sum(quantity) AS nr_products
FROM order_details
GROUP BY order_id;
```

Aggregering i grupper: Eksempel 2

Finn gjennomsnittspris for hver kategori (i Northwind)

```
SELECT c.category_name, avg(p.unit_price) AS Averageprice
FROM categories AS c INNER JOIN products AS p
ON (c.category_id = p.category_id)
GROUP BY c.category_name;
```

Grupper på flere kolonner:

- Vi kan også gruppere på flere kolonner
- Da vil hver gruppe bestå av de radene med like verdier på alle kolonnene vi grupperer på

Finn antall produkter for hver kombinasjon av kategori og hvorvidt produktet fortsatt selges

```
SELECT c.category_name, p.discontinued, count(*) AS nr_products
FROM categories AS c INNER JOIN products AS p
ON (c.category_id = p.category_id)
GROUP BY c.category_name, p.discontinued;
```

Aggregering i grupper: Eksempel 3

Finn navn på ansatte og antall bestillinger den ansatte har håndtert, sortert etter antall bestillinger fra høyest til lavest

```
SELECT format('%s %s', e.first_name, e.last_name) AS emp_name,
count(*) AS num_orders
FROM orders AS o INNER JOIN employees AS e
ON (o.employee_id = e.employee_id)
GROUP BY e.first_name, e.last_name
ORDER BY num_orders DESC;
```

Hvis vi kun vil ha personer med forskjellige navn:

```
SELECT format('%s %s', e.first_name, e.last_name) AS full_name, count(*) AS  
nr_orders
```

```
GROUP BY e.employee_id, e.first_name, e.last_name  
ORDER BY nr_orders DESC;
```

Filtrere på aggregat-resultat:

- I enkelte tilfeller er vi kun interessert i grupper hvor et aggregat har en bestemt verdi
- F.eks dersom man vil vite kategorinavn og antall produkter på de kategoriene som har flere enn 10 produkter
- Nå kan vi gjøre dette med en delspørring:

```
SELECT category_name, nr_products  
FROM (  
    SELECT c.category_name, count(*) AS nr_products  
    FROM categories AS c  
        INNER JOIN products AS p ON (c.category_id = p.category_id)  
    GROUP BY c.category_name) AS t  
WHERE nr_products > 10;
```

- Men det finnes en egen klausul for å vege ut grupper

HAVING-klausulen:

- Denne klausulen heter HAVING og kommer rett etter GROUP BY, slik:

```
SELECT c.category_name, count(*) AS nr_products  
FROM categories AS c  
        INNER JOIN products AS p ON (c.category_id = p.category_id)  
GROUP BY c.category_name  
HAVING count(*) > 10;
```

- Merk: kan ikke bruke navnene vi gir i SELECT
- HAVING blir altså som en slags WHERE for grupper

Oversikt over SQLs SELECT

- ◆ Vi har nå sett mange nye klausuler
- ◆ Generelt ser våre SQL-spørringer nå slik ut:

```
WITH <navngitte-spørringer>
SELECT <kolonner>
FROM <tabeller>
WHERE <uttrykk>
GROUP BY <kolonner>
HAVING <uttrykk>
ORDER BY <kolonner> [DESC]
LIMIT <N>
OFFSET <M>
```

Forklaring:

- WITH <navngitte-spørringer> —> For å navngi delspørringer
- SELECT <kolonner> —> For å hente ut og manipulere kolonner
- FROM <tabeller> —> For å joine sammen tabeller
- WHERE <uttrykk> —> Hvor vi kan begrense hvilke rader vi er interessert i
- GROUP BY <kolonner> —> For å gruppere rader
- HAVING <uttrykk> —> Som er en slags WHERE-klausul for grupper
- ORDER BY <kolonner> [DESC] —> For å sortere output
- LIMIT <N> —> For å begrense antall rader i output
- OFFSET <M> —> For å hoppe over rader

Det skal i SELECT-spørringer alltid være i rekkefølgen over, men LIMIT og OFFSET kan byttes plass på hvis det trengs.

Kan selvfølgelig droppe klausuler, men må ha GROUP BY for å ha HAVING

Avanserte eksempler:

Enklere syntaks for joins

- ◆ Man kan bruke **USING** (<kolonne>) fremfor **ON** (a.<kolonne> = b.<kolonne>)
- ◆ For eksempel:

```
SELECT p.product_name, c.category_name
FROM products AS p
      INNER JOIN categories AS c USING (category_id);
```

- ◆ Merk: Må fortsatt bruke **ON** dersom kolonnene har ulikt navn

Eksempel: Variable i delspørringer (1)

Finn navnet på alle produkter som har lavere pris nå enn gjennomsnittsprisen den er solgt for tidligere [1 rad]

```
SELECT p.product_name
FROM products AS p
WHERE p.unit_price <
      (SELECT avg(d.unit_price)
       FROM order_details AS d
       WHERE p.product_id = d.product_id);
```

Merk: Man kan bruke variabler fra en spørring i dens delspørringer

Eksempel: Variable i delspørringer (2)

Finn antall produkter for hver kategori

```
SELECT c.category_name,
       (SELECT count(*)
        FROM products AS p
        WHERE p.category_id = c.category_id) AS nr_products
FROM categories AS c
```

Utlede informasjon om entiteter:

- Aggregering i grupper, sortering og å begrense svaret er svært nyttig når man har store mengder data
- Når vi grupperer kan vi enten utlede implisitt informasjon om allerede eksisterende entiteter
- Eller lage nye entiteter fra attributter
- Sortering og begrensning lar oss hente ut de mest interessante objektene
- Dette gjør også at vi kan lage langt mer interessante views

Eksempel 1: Implisitt informasjon om kategorier

For hver kategori, finn høyeste, laveste, og gjennomsnittspris på produktene i kategorien, samt antall produkter

```
SELECT c.category_id, c.category_name,
       max(p.unit_price) AS highest,
       min(p.unit_price) AS lowest,
       avg(p.unit_price) AS average,
       count(*) AS nr_products
FROM categories AS c
     INNER JOIN products AS p USING (category_id)
GROUP BY c.category_id, c.category_name;
```

Eksempel 2: Implisitt informasjon om land

Finn de tre mest kjøpte produktene for hvert land

```
WITH
  bought_by_country AS (
    SELECT c.country, p.product_name, count(*) AS nr_bought
    FROM products AS p
         INNER JOIN order_details AS d USING (product_id)
         INNER JOIN orders AS o USING (order_id)
         INNER JOIN customers AS c USING (customer_id)
    GROUP BY c.country, p.product_name
    ORDER BY nr_bought DESC
  ),
  countries AS (
    SELECT DISTINCT country FROM customers
  )
SELECT c.country,
       (SELECT s.product_name FROM bought_by_country AS s
        WHERE s.country = c.country
        LIMIT 1) AS first_place,
       (SELECT s.product_name FROM bought_by_country AS s
        WHERE s.country = c.country
        OFFSET 1
        LIMIT 1) AS second_place,
       (SELECT s.product_name FROM bought_by_country AS s
        WHERE s.country = c.country
        OFFSET 2
        LIMIT 1) AS third_place
FROM countries AS c;
```

Anbefalingssystem (Komplisert eksempel! Utenfor pensum)!!!!!!!!!!!!!!

Vi vil lage en spørring som finner ut:

- Hvilke produkter vi kan anbefale en kunde å kjøpe,
- Basert på hva kunden har kjøpt,
- Og hva andre kunder som har kjøpt det samme har kjøpt.

Anbefalingssystem (Komplisert eksempel! Utenfor pensum)

```
WITH
  bought AS ( -- Relaterer kunde-IDer til produkt-IDene til det de har kjøpt
    SELECT DISTINCT c.customer_id, d.product_id -- Vil ikke ha duplikater!
    FROM customers AS c
      INNER JOIN orders USING (customer_id)
      INNER JOIN order_details AS d USING (order_id)
  ),
  correspondences AS ( -- Relaterer par av produkter til antallet ganger disse er kjøpt av samme kunde
    SELECT b1.product_id AS prod1, b2.product_id AS prod2, count(*) AS correspondence
    FROM bought AS b1
      INNER JOIN bought b2 USING (customer_id)
    WHERE b1.product_id != b2.product_id -- Fjern par hvor produktene er like
    GROUP BY b1.product_id, b2.product_id -- Grupper på par av produkter
    HAVING count(*) > 18 -- Antall korrespondanser bør være litt høyt
  ),
  reccomend AS ( -- Relaterer kunde-IDer til anbefalte produkters IDer
    SELECT DISTINCT b.customer_id, c.prod2 AS product_id -- Vil ikke ha duplikater!
    FROM correspondences AS c
      INNER JOIN bought AS b
      ON (b.product_id = c.prod1)
    WHERE NOT c.prod2 IN
      (SELECT product_id
       FROM bought AS bi
       WHERE b.customer_id = bi.customer_id)
  )
-- Til slutt finn navn på både kunde og produkt og aggreger produktnavnene
-- for hver kunde i ett array med aggregatfunksjonen array_agg
SELECT c.company_name, array_agg(p.product_name) AS reccomended_products
FROM customers AS c INNER JOIN reccomend AS r USING (customer_id)
  INNER JOIN products AS p USING (product_id)
GROUP BY c.customer_id, c.company_name;
```

UKE 42 Avansert SQL

Eksempler: Aggregering og sortering

Avansert SQL: Oppgave 6

Finn de 10 skuespillerne som har spilt i flest filmer, men som har spilt i filmer med gjennomsnittsrating høyere enn 9, sortert etter antall filmer de har spilt i

```
WITH
  played_in AS (
    SELECT DISTINCT fp.personid, fp.filmid, r.rank
    FROM filmparticipation AS fp
      INNER JOIN film AS f USING (filmid)
      INNER JOIN filmrating AS r USING (filmid)
    WHERE fp.parttype = 'cast'
  )
SELECT p.personid, p.firstname, p.lastname, count(*) AS nr_played_in
FROM person AS p
  INNER JOIN played_in AS pi USING (personid)
GROUP BY p.personid, p.firstname, p.lastname
HAVING avg(pi.rank) > 9
ORDER BY nr_played_in DESC
LIMIT 10;
```

UKE 43

Ytre joins:

Aggregering og NULL:

- Aggregering med sum, min, max og avg ignorerer NULL-verdier
- Det betyr også at dersom det kun er NULL-verdier i en kolonne blir resultatet av disse NULL
- Count(*) teller med NULL-verdier
- Men dersom vi oppgir en konkret kolonne, f.eks count(product_name) vil den kun telle verdiene som ikke er NULL

OUTER JOINS:

- Vi har flere varianter av ytre joins, nemlig
 - Left outer join
 - Right outer join
 - Full outer join

- Brukes ved å bytte ut INNER JOIN med f.eks LEFT OUTER JOIN
- Hovedidéen bak denne typen join er å bevare alle rader fra en eller begge tabellene i joinen
- Og så fylle inn med NULL hvor vi ikke har noen match

LEFT OUTER JOIN:

- I en LEFT OUTER JOIN vil alle rader i den venstre tabellen bli med i svaret
- Resultatet av a LEFT OUTER JOIN b ON (a.c1 = b.c2) blir
 - Samme som a INNER JOIN b ON (a.c1 = b.c2),
 - Men hvor alle rader fra a som ikke matcher noen i b
 - (Altså hvor a.c1 ikke er lik noen b.c2)
 - Blir lagt til resultatet, med NULL for alle bs kolonner

RIGHT OUTER JOIN:

- Akkurat samme som over, men på motsatt side
- A RIGHT OUTER JOIN b on (a.c1 = b.c2) er det samme som: b LEFT OUTER JOIN a ON (a.c1 = b.c2)

FULL OUTER JOIN:

- Det er en slags kombinasjon av de to over, her vil ALLE rader være med i svaret

UKE 44

Programmering med SQL

Programmering med databaser:

- Som oftest er det ikke mennesker som manuelt skriver SQL
- Men programmerer som generer spørringer som de sender til databasen
- Spørringene kan da genereres basert på bruker-input, hendelser, osv.

Eksempel:

Går inn på <https://finn.no/s> «bolig til salgs» og setter:

- Sted: Oslo eller Akershus
- Makspris: 5,000,000,-
- Minpris: 3,000,000,-
- Antall rom: 3

Og klikker «søk».

Generert SQL-spørring:

```
SELECT *
```

```
FROM boliger
```

```
WHERE (sted = 'Oslo' OR sted = 'Akershus') AND pris <= 5000000 AND pris > 3000000 AND ant_rom >= 3;
```

Altså skriver ikke brukere selv SQL spørringer, men det blir generert av programmereren.

Generelle prinsipper:

- Programmerer håndterer SQL-spørringer som strenger
- Kan dermed manipulere SQL-spørringer akkurat som strenger
- For å kunne sende en spørring til en database trenger man to ting:
 - En tilkobling - Connection
 - En eller flere spørrings-ersekverere - Cursor/Statement

Connection:

- Connection-objekter er ansvarlige for å lage en tilkobling til databasen
- Input til disse er databasenavn, brukernavn, passord, host osv.
- Når tilkoblingen er vellykket kan man begynne å lage spørrings-ersekverere fra en Connection

Spørrings-eksekverere:

- Lages fra en Connection
- Gis en spørring som en streng
- Kan så eksekvere spørringen via et metode-kall (typisk execute())
- Kan så hente ut svarerne fra spørringen

Java og JDBC (Java Database Connection):

- Biblioteket for interaksjon med databaser fra Java heter JDBC
- Egen driver for PostgreSQL som lastes inn med `Class.forName(«org.postgresql.Driver»)`
- Kan lage Connection-objekt ved å kalle `DriverManager.getConnection(<conStr>)` hvor <conStr> er en streng som inneholder en URI med tilkoblingsdetaljer
- Kan så lage Statement/PreparedStatement-objekter ved å kalle `connection.createStatement()` eller `connection.prepareStatement()`
- En spørring eksekveres ved å kalle `statement.execute()`
- Resultatene fra en spørring kommer i form av et `ResultSet`

ResultSet:

- Et `ResultSet` holder alltid en peker til én rad i resultatet
- Man kan hoppe videre til neste rad ved å kalle metoden `next()`
- Denne metoden returnerer en boolsk verdi som er usann dersom det ikke finnes flere rader i resultatet
- Man kan bruke `next()` i en WHILE-løkke for å vite at det finnes en neste rad, men man må bruke `next()`-metoden en gang først for å komme til den første linjen
- For hver mulige type har man en egen get-metode (f.eks `getString()`, `getInt()`) som tar en int som argument som er kolonne-nummeret
- Så `result.getString(2)` henter ut verdien i kolonne 2 i den nåværende raden, som en streng

WEBSHOP-EKSEMPEL

Webshop:

- Vi skal lage programmer for en nettbutikk
- Skal i videoene lage program som lar brukere
 - Registrere ny bruker
 - Logge inn
 - Søke etter produkter
 - Bestille produkter

- I boligen skal dere lage to programmer
 - Ett som lar ansatte legge inn nye kategorier og produkter
 - Ett som lager regninger for kunder

Logge seg inn i databasen med webshop.sql filen:

1. Kom deg inn i mappen som har filen
2. `Psql -h dbpg-ifi-kurs01 -d leifhka -U leifhka_priv`
3. Skriv inn passordet
4. `\i webshop.sql`
5. Kan i tillegg skrive `\i data.sql` for å få inn ekstra data i SCHEMA-et
6. `\dt ws.*`
7. `SELECT * FROM ws.users;`

Notater uke 45:

Sikkerhet i databaser:

Hovedmål med databasesikkerhet:

- Konfidensialitet
 - Uvedkommende må ikke kunne se data de ikke skal ha tilgang til.
- Integritet
 - Data må være korrekte og pålitelige. Derfor må data beskyttes mot endringer fra uautoriserte brukere
- Tilgjengelighet
 - Brukere må kunne se eller modifisere data de har fått tilgang til

Sikkerhet: Ikke bare i databasesystemet

- Programmer inneholder ofte mye mer enn bare databasen
- Databasesikkerhet kan derfor ikke kun fokusere på databasen
- Sikkerhetshull kan forekomme i alle ledd (frontend, backend, nettverket, osv.)
- Sikkerhet er derfor alltid en helhetlig oppgave
- Må derfor sikre hver enkelt komponent og interaksjonen mellom dem
- Må være tydelige på hvilke antagelser om andre komponenter hver del av systemet gjør

Tilgangskontroll: Klient/frontend. Her er det klienten som autoriserer tilgangskontrollen

- Klienten autentiserer brukerne
- Klienten sjekker hva brukeren har lov til
- Backend og databasen må stole på at klienten gjør dette riktig
- Trenger derfor sikring slik at bare klienten kan få tilgang til backend/databasen

Tilgangskontroll: Backend. Her er det backend-programmet som autoriserer tilgangskontrollen

- Backend autentiserer brukerne
- Backend sjekker hva brukeren har lov til
- Backend har ofte én bruker til databasen
- Databasen må stole på at backend gjør jobben riktig
- Backend og databasen er ofte bak samme brannmur

Tilgangskontroll: Database. Her er det databasesystemet som autoriserer tilgangskontrollen

- Databasesystemet autentiserer brukerne direkte
- Hver bruker av programmet får da hver sin databasebruker
- Klienten kan forhåndssjekket (f.eks. for å tilpasse brukergrensesnittet)
- Til syvende og sist så er det databasen som har brukerne lagret. Klienten og backenden har brukere i databasesystemet

Tilgangskontroll i databaser:

- Tilgang til databasen kontrolleres gjennom tre ting:
 - Brukere = 1
 - Roller = 2
 - Rettigheter = 3
- For eksempel:
 - Bruker philipef(1) har rollen kundeadmin(2)
 - Bruker philipef(1) har rollen produktansvarlig(2)
 - Bruker philipef(1) har rollen kunde(2)
 - Brukere med rollern kundeadmin(2) kan «opprette og oppdatere kunder (rader i customer-tabellen)»(3)
 - Brukere med rollen produktansvarlig(2) kan «opprette, oppdatere og slette produkter (rader i products-tabellen)»(3)

Brukere vs. Roller:

- Mulig å gi hver bruker de rettighetene de skal ha
- Men vanskelig å holde rede på at hver bruker har de riktige rettighetene
- Spesielt om det er mange brukere og mange rettigheter
- Typisk vil mange brukere trenge samme rettigheter: Vanskelig å vedlikeholde
- Vi lager derfor roller som fanger en mengde med rettigheter som hører sammen
- Og gir deretter brukere de passende rollene
- Dette heter Role-based Access Control

Databasebrukere:

- F.eks når dere logger dere inn i databasen med: `$ psql -h dbpg-ifi-kurs03 -U philipef -d fdb` er philipef brukeren
- Autentisering skjer typisk via passord, SSH public keys, el.
- Gyldige brukernavn og (krypterte) passord lagres av RDBMS (Relational Database Management System)
- Autentisering kan delegeres til andre systemer. Når vi bruker psql kommandoen over, så delegeres det til feide.
- Alle databaser, skjema, tabeller, views, osv. eies av en bruker

Lage brukere og roller med SQL:

- For å lage en ny bruker philipef med passord passord123 og rollene kundeadmin og produktansvarlig kan man kjøre følgende SQL-kommando:
 - `CREATE USER philipef WITH PASSWORD 'passord123'`
 - `ROLE kundeadmin, produktansvarlig;`
- Roller lages nesten helt likt:
 - `CREATE ROLE produktansvarlig;`
- I PostgreSQL er `CREATE USER` bare et alias for `CREATE ROLE` med LOGIN-adgang
- Roller og brukere kan slettes med `DROP`

Begrense bruk:

- Kan begrense hvor lenge en bruker eller rolle skal være gyldig ved å sette `VALID UNTIL '2021-01-01'` i kommandoene

- Kan begrense antall tilkoblinger en bruker/rolle kan ha ved å sette CONNECTION LIMIT 5 i kommandoene
- Dette er det som gjør at neon av dere har fått feilmeldingen:
psql: FATAL: too many connections for role 'philipef'
- For å gi en bruker/rolle (generelle) rettigheter til å lage databaser, roller, osv.
Kan man legge til CREATEDB, DREATEROLE, osv.

Gi og fjerne rettigheter:

- Man kan gi roller/brukere mer detaljerte rettigheter via GRANT-kommandoen
- GRANT-kommandoen har følgende form:
FRANT <privileges> ON <object> TO <role>;
- Hvor <privileges> f.eks:
SELECT, UPDATE, INSERT, DELETE, CREATE, CONNECT, USAGE, ALL
- Gir man rettigheter til en rolle, vil alle dens medlemmer også få disse
- Fjerning av rettigheter kan gjøres tilsvarende med REVOKE

GRANT-eksempler:

- For å gi rollen kundeadmin rettighetene til å opprette og oppdatere ws.users-tabellen kan vi kjøre følgende kommando:
GRANT INSERT; UPDATE ON TABLE ws.users TO kundeadmin;
- For å gi rollen webshopadmin alle rettigheter innenfor skjemaet ws:
GRANT ALL ON SCHEMA ws TO webshopadmin;
- Kan også gi en bruker en ny rolle med GRANT:
GRANT kundeadmin TO philipef;
- Kan til og med gi tillatelser på kolonnenivå. Altså kan prisansvarlig endre prisene, men ikke navn, produkter osv.:
GRANT UPDATE (Price) ON ws.products TO prisansvarlig;
- For å fjerne kunde-rollens tilgang til categories kan vi kjøre:
REVOKE USAGE ON ws.categories FROM kunde;

Tilgang og views:

- I enkelte tilfeller ønsker vi ikke gi tilgang til tabellene direkte
- Men d.eks kun aggregerte eller utvalgte verdier
- F.eks vil ikke gi tilgang til pasientjournalen til hver enkelt person, men heller antall med ulike sykdommer per kommune
- Kan da lage views, og så gi tilgang til disse

Sikkerhet i programmer som bruker databaser:

Databasetilkoblinger:

- Connection-objekter (og de tilhørende cursor eller Statement og ResultSet) er ressurser som kan føre til minnelekasjer
- Alle disse har en egen metode som heter close() som stenger ressursen og frigjør den
- For de enkle programmene vi så sist uke var dette ikke veldig viktig, ettersom ressursene blir frigjort når programmet avsluttes
- Men for større programmer og tjenester som skal kjøre over lengre tid bør man alltid sørge for at tilkoblingene blir stengt med en gang man er ferdige med dem.

Stenge tilkoblinger:

- I Python har både Connection og Cursor en close() metode
- I Java har både Connection, Statement/PreparedStatement og ResultSet egne close() metoder
- Generelt blir alle objekter stengt når objektet det er laget fra stenges
- F.eks vil en PreparedStatement stenges dersom dens Connection stenges

Automatisk stenge ressurser:

- Java kan bruke try-with-blokker for automatisk å stenge tilkoblinger

SQL injections:

- SQL injections er en type angrep mot en klient/backend hvor en (ondsinnet) bruker får kjørt egen SQL-kode på databasen
- Brukeren kan da forbigå autentisering eller hente ut, endre eller slette data brukeren egentlig ikke skal ha tilgang til
- SQL injection utnytter følgende faktorer:
 - At vi har et program som kjører SQL-spørringer (F.eks Python eller Java)
 - Programmet tar input fra brukeren som skal være en del av spørringene

- Programmet ikke skiller mellom data og kommandoer i spørringen

SQL injections: Grunnleggende prinsipp:

- La oss si at en nettbutikk lar brukere søke etter varer på følgende måte:
product = input(«Produkt: «);
cur.execute(«SELECT * FROM products WHERE navn = '« + product + «');
- Med input Socks blir spørringen:
SELECT * FROM products WHERE navn = 'Socks';
- Med input O´boy får vi ERROR:
SELECT * FROM products WHERE navn = 'O´Boy';
Grunnen til at det blir feil er fordi selve variabelen er ferdig på O-en.
- Ved å bruke denne typen SQL spørring for input fra brukeren, så oppstår det en sårbarhet (SQL injection):
Med input: Socks'; DROP TABLE products;— Får vi:
SELECT * FROM products WHERE navn = 'Socks'; DROP TABLE products;
—';
Her blir altså alle feilmeldinger og annen SQL kode ikke tatt med og hele tabellen blir fjernet av angrepet til brukeren.

Hvorfor er dette viktig?:

- En av de vanligste formene for hackerangrep
- Dersom angrepet lykkes vil det gi den ondsinnede brukeren veldig mye makt:
 - Ødelegge/slette data
 - Endre data
 - Hente ut konfidensiell informasjon
 - Hindre tjenesten i å fungere (DOS)
- Veldig enkelt å forhindre med parametrisere spørringer!

Helhetlig sikkerhet:

- Ha Placeholders i koden for å forhindre SQL-injections. Hvis en da skriver det som ble gjort over, så vil det heller bare komme SQL kommandoer i for eksempel adressen.
- Merk at selvom klienten skulle være sårbar for SQL injection-angrep kan databasen likevel forhindre mye skade om man har satt riktige rettigheter
- F.eks dersom databasebrukeren som klienten benytter ikke har tilgang til å slette eller endre tabeller eller spørre mot vilkårlige tabeller
- Dette viser hvor viktig det er at alle ledd er sikret og at man ikke bør anta for mye av andre komponenter i et system.

Notater uke 46:

Indekser:

Spørringer og kompleksitet:

- Hvordan utfører en database et oppslag på en bestemt verdi?
- Eller en join mellom to tabeller?
- Begge disse problemene er egentlig et søk etter bestemte verdier i en kolonne
- For at databasen skal kunne utføre disse operasjonene effektivt (spesielt over veldig store tabeller) trenger vi datastrukturer som gjør søket mer effektivt
- Slike datastrukturer heter indeksstrukturer

Indeksstrukturer:

- En indeksstruktur er en datastruktur som lar databasen hurtig finne bestemte rader i en tabell, basert på verdiene i en (eller flere) kolonner
- Husk at databaser lagrer data på disk, ikke i minne (RAM)
- Databaseindekser skiller seg litt fra andre datastrukturer fordi de ikke lagres i minne, men på disk

Lese fra harddisk:

- Å lese fra disk tar ca. 10, 000 ganger lengre tid enn fra RAM (avhengig av disktype og minnetype)
- Indeksstrukturer i databaser er derfor optimert for å utføre så få diskoppslag som mulig
- Har to hovedtyper indekser: Hash-baserte og tre-baserte

B-tre-indekser:

- Trestruktur hvor hver node kan ha mange barn
- Nodene har samme størrelse som en disk-blokk

- Minimerer antall oppslag på disk
- Hver verdi i løvnodene har pekere til dens tilhørende rad i den tilhørende tabellen
- Kan utføre effektive oppslag på konkrete verdier
- Samt effektive intervall søk

Hash.indekser:

- En hash-indeks bruker en hash-funksjon for å oversette en verdi til en minneadresse
- På minneadressen ligger så en liste med pekere til rader som har denne verdien
- Hash-indekser er mer effektive på oppslag på konkrete verdier
- Men kan ikke brukes for intervaller (må da gjøre ett oppslag for hver mulige verdi i intervallet)

Nøkler og indekser:

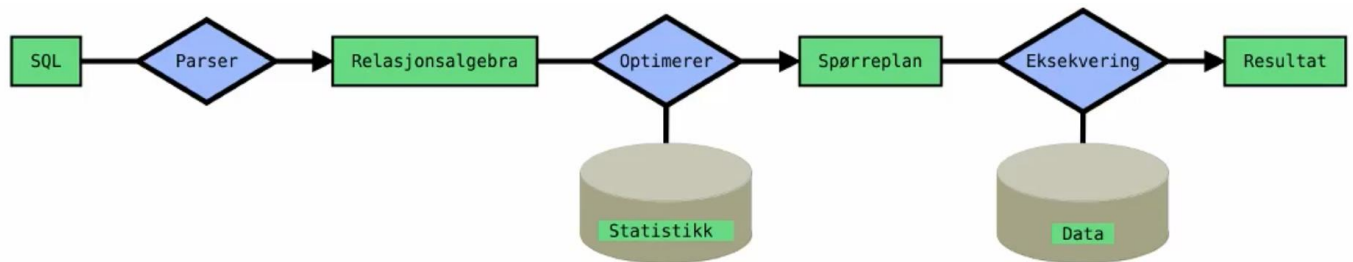
- Når man markerer en kolonne med PRIMARY KEY blir det automatisk opprettet en B-tre-indeks på denne kolonnen
- Joins over primærnøkler er derfor alltid relativt effektive
- Men, det kan hende man ønsker å gjøre søk, oppslag eller joins over kolonner som ikke er primærnøkler
- Vi må da lage indeksene selv

Lage indekser med SQL:

- For å lage en indeks på en kolonne trenger man bare å skrive:
`CREATE INDEX <index_name> ON <table>(<columns>);`
 hvor <index_name> er navnet på indeksen, <table> er et tabellnavn og <columns> er en liste med kolonner man ønsker å indeksere
- Databasen lager da en passende indeks (typisk B-tre)
- F.eks, hvis vi vil lage en indeksstruktur over unit_price kolonnen:
`CREATE INDEX price_index ON products(unit_price);`
- Merk: Dersom man lister opp flere kolonner blir indeksen over alle kolonnene samtidig
- Databasen finner selv ut når indeksen bør brukes

Spørreprosessering:

Spørreprosessering: Oversikt:



- En spørring går gjennom flere steg før den til slutt blir evaluert over dataene
- Disse stegene sørger for at spørringen blir besvart så effektivt som mulig
- Parsing: Det første steget er at spørringen sjekkes syntaktisk (f.eks tabellene og kolonnene finnes i databasen, typene er riktige osv.)
Hvis det er noen feil med SQL-spørringen blir det kastet en ERROR melding og avbrutt.
- Relasjonsalgebra: Hvis det ikke er noen feil i SQL-spørringen, så blir det deretter oversatt til et spørre-tre over relasjons algebraen
- Optimierer: Spørringen uttrykt i relasjonell algebra kan manipuleres algebraisk
Dette brukes for å generere forskjellige men ekvivalente spørringer
Altså, spørringer som gir samme svar, men ser forskjellige ut
Forskjellige spørringer kan ha ulik kompleksitet
De ulike spørringene skal så (i neste steg) bli tilordnet en ca. kostnad (vil si hvor lang tid spørringen vil ta)
Vi vil så velge den spørringen som er billigst å eksekvere

Optimerer: Fra spørring til kostnad:

- De ulike spørringene blir så tilordnet en kostnad
- Kostnadsevalueringen bruker statistikk over databasen
- F.eks antall rader i hver tabell, antall ulike verdier i hver kolonne, osv.
- Bruker her også skranker (f.eks UNIQUE, CHECK) og indeksstrukturer
- Høyere kostnad betyr lengre eksekveringstid
- Databasen velger så den spørringen med lavest kostnad
- Det siste som skjer i Optimierer-steget er at det blir laget en spørreplan for den valgte spørringen
- Dette er en mer detaljert plan for hvordan spørringene skal eksekveres

Eksekvering: Evaluering:

- Til slutt evalueres spørringen over databasen
- Databasen har så svært effektive algoritmer for joins, oppslag, sortering, osv.

- Merk: Databasen trenger kun én algoritme per operator i den (utvidede) relasjonelle algebraen

Notater uke 47: